EVALUACIÓN DEL SEGUNDO PARCIAL
BASES DE DATOS PARA CÓMPUTO EN LA NUBE
ADMINISTRACIÓN DE BASES DE DATOS PARA CÓMPUTO EN LA NUBE

Nombre: N a i r d a T o n a n t z i n M o r a l e s D e l R e a l	
Carrera: TSU en Tecnologías de la Información, Área Desarrollo de Software Multiplataforma.	**Grupo:** 5ATI
Asignatura: Bases de Datos para Cómputo en la Nube	**Unidad:** II Sistemas Gestores de Bases de Datos NoSQL, III Administración de bases de datos NoSQL.
Profesor: MGTI Jorge Luis De La Paz Ramos	

Título: “Consultas NoSQL”

Instrucciones: en MongoDB, crear la base de datos “créditos”, esta se describe en la siguiente imagen, insertar los datos de las tablas en las colecciones que conformarán a la base de datos mediante sentencias insertOne e insertMany.

Después de la siguiente imagen, agregar a este documento los códigos de la creación de la base de datos, creación de las colecciones e inserción de documentos. (Imágenes o texto).

tipo_cli

_id	nom_t	estatus
1	Inicial	0
2	Frencuente	1
3	Distinguido	1

cliente

_id	cod_cli	nombres	apaterno	amaterno	sexo	edad	id_t	estatus
1	CLI-001	José	López	Pérez	M	19	1	1
2	CLI-002	Daniela	Ibarra	Aguilar	F	30	2	0
3	CLI-003	Mario	Sánchez	Morales	M	26	2	0
4	CLI-004	Alberto	Osako	Murillo	M	29	1	0
5	CLI-005	Gabriela	Sandoval	Mendoza	F	40	3	0
6	CLI-006	Adriana	Aguilera	Sandoval	F	18	2	1
7	CLI-007	Eugenia	Guerrero	Díaz	F	45	1	1
8	CLI-008	Armando	Ramos	García	M	42	3	1
9	CLI-009	Adriana	Sánchez	Moncada	F	33	2	1
10	CLI-010	Eva	Ramos	Solis	F	50	1	0

solicitud_cred

_id	cod_sol	monto	id_c	plazo_pago_meses	estatus
1	SOL-001	50000	10	12	1
2	SOL-002	45000	5	24	2
3	SOL-003	20000	4	36	2
4	SOL-004	10000	1	48	1
5	SOL-005	100000	2	60	3
6	SOL-006	30000	3	36	2
7	SOL-007	90000	7	24	1
8	SOL-008	15000	6	3	3
9	SOL-009	100000	9	3	2
10	SOL-010	40000	8	5	1

tipo_estatus_c

_id	nom_tipo_ep
1	Aceptado
2	Rechazado
3	En proceso

EVALUACIÓN DEL SEGUNDO PARCIAL
BASES DE DATOS PARA CÓMPUTO EN LA NUBE
ADMINISTRACIÓN DE BASES DE DATOS PARA CÓMPUTO EN LA NUBE

Creación BD: use créditos

Creación e inserción de las colecciones:

Colección tipo_cli:

```
creditos> db.tipo_cli.insertOne({ _id: 1, nom_t: "Inicial", estatus: 0 });
creditos> db.tipo_cli.insertOne({ _id: 2, nom_t: "Frecuente", estatus: 1 });
creditos> db.tipo_cli.insertOne({ _id: 3, nom_t: "Distinguido", estatus: 1 });
```

Colección cliente:

```
creditos> db.cliente.insertOne({ _id: 1, cod_cli: "CLI-001", nombres: "José", apaterno: "López", amaterno: "Pérez", sexo
: "M", edad: 19, id_t: 1, estatus: 1 });
creditos> db.cliente.insertOne({ _id: 2, cod_cli: "CLI-002", nombres: "Daniela", apaterno: "Ibarra", amaterno: "Aguilar"
, sexo: "F", edad: 30, id_t: 2, estatus: 0 });
creditos> db.cliente.insertOne({ _id: 3, cod_cli: "CLI-003", nombres: "Mario", apaterno: "Sánchez", amaterno: "Morales"
 sexo: "M", edad: 26, id_t: 2, estatus: 0 });
creditos> db.cliente.insertMany([
...     { _id: 4, cod_cli: "CLI-004", nombres: "Alberto", apaterno: "Osako", amaterno: "Murillo", sexo: "M", edad: 29, i
id_t: 1, estatus: 1 },
...     { _id: 5, cod_cli: "CLI-005", nombres: "Gabriela", apaterno: "Sandoval", amaterno: "Mendoza", sexo: "F", edad: 4
0, id_t: 3, estatus: 0 },
...     { _id: 6, cod_cli: "CLI-006", nombres: "Adriana", apaterno: "Aguilera", amaterno: "Sandoval", sexo: "F", edad: 1
8, id_t: 2, estatus: 1 },
...     { _id: 7, cod_cli: "CLI-007", nombres: "Eugenia", apaterno: "Guerrero", amaterno: "Díaz", sexo: "F", edad: 45, i
id_t: 1, estatus: 1 },
...     { _id: 8, cod_cli: "CLI-008", nombres: "Armando", apaterno: "Ramos", amaterno: "García", sexo: "M", edad: 42, id
id_t: 3, estatus: 1 },
...     { _id: 9, cod_cli: "CLI-009", nombres: "Adriana", apaterno: "Sánchez", amaterno: "Moncada", sexo: "F", edad: 33,
 id_t: 2, estatus: 1 },
...     { _id: 10, cod_cli: "CLI-010", nombres: "Eva", apaterno: "Ramos", amaterno: "Solís", sexo: "F", edad: 50, id_t:
1, estatus: 0 }
... ]);
```

Colección solicitud_cred:

```
creditos> db.solicitud_cred.insertOne({ _id: 1, cod_sol: "SOL-001", monto: 50000, id_c: 10, plazo_pago_meses: 12, estatu
estatus: 1 });
creditos> db.solicitud_cred.insertOne({ _id: 2, cod_sol: "SOL-002", monto: 45000, id_c: 4, plazo_pago_meses: 24, estatus
estatus: 2 });
creditos> db.solicitud_cred.insertOne({ _id: 3, cod_sol: "SOL-003", monto: 20000, id_c: 1, plazo_pago_meses: 36, estatus
estatus: 2 });
creditos> db.solicitud_cred.insertMany([
...     { _id: 4, cod_sol: "SOL-004", monto: 100000, id_c: 2, plazo_pago_meses: 48, estatus: 1 },
...     { _id: 5, cod_sol: "SOL-005", monto: 100000, id_c: 3, plazo_pago_meses: 60, estatus: 3 },
...     { _id: 6, cod_sol: "SOL-006", monto: 30000, id_c: 5, plazo_pago_meses: 36, estatus: 2 },
...     { _id: 7, cod_sol: "SOL-007", monto: 90000, id_c: 7, plazo_pago_meses: 24, estatus: 1 },
...     { _id: 8, cod_sol: "SOL-008", monto: 15000, id_c: 6, plazo_pago_meses: 3, estatus: 3 },
...     { _id: 9, cod_sol: "SOL-009", monto: 100000, id_c: 9, plazo_pago_meses: 3, estatus: 2 },
...     { _id: 10, cod_sol: "SOL-010", monto: 40000, id_c: 8, plazo_pago_meses: 5, estatus: 1 }
... ]);
{
```

Colección tipo_estatus_c:

```
creditos> db.tipo_estatus_c.insertOne({ _id: 1, nom_tipo_ep: "Aceptado" });
creditos> db.tipo_estatus_c.insertOne({ _id: 2, nom_tipo_ep: "Rechazado" });
creditos> db.tipo_estatus_c.insertOne({ _id: 3, nom_tipo_ep: "En proceso" });
```

EVALUACIÓN DEL SEGUNDO PARCIAL
BASES DE DATOS PARA CÓMPUTO EN LA NUBE
ADMINISTRACIÓN DE BASES DE DATOS PARA CÓMPUTO EN LA NUBE

Una vez creada la base de datos “créditos” y con los documentos registrados, realizar las siguientes consultas. Después del enunciado de cada consulta, para comprobar la ejecución y resultados es necesario mostrar el código correspondiente y la imagen del resultado. (El código puede ser en imagen o texto).

1.- Actualizar a “1” el estatus de los documentos de la colección tipo_cli que se encuentren en “0”.
Código:

```
creditos> db.tipo_cli.updateMany(
...     { estatus: 0 },
...     { $set: { estatus: 1 } }
... )
...
{
```

Resultado:

2.- Mostrar la cantidad de documentos que se encuentran registrados en la colección cliente.
Código:

```
creditos> db.cliente.countDocuments()
```

Resultado:

```
10
```

3.- Actualizar a “1” el estatus de los documentos de la colección cliente que se encuentran “0” y el sexo es “M”.
Código:

```
creditos> db.cliente.updateMany(
...     { estatus: 0, sexo: "M" },
...     { $set: { estatus: 1 } }
... )
```

Resultado:

```
creditos> db.cliente.find({sexo:"M"})
[
  {
    _id: 1,
    cod_cli: 'CLI-001',
    nombres: 'José',
    apaterno: 'López',
    amaterno: 'Pérez',
    sexo: 'M',
    edad: 19,
    id_t: 1,
    estatus: 1
  },
  {
    _id: 3,
    cod_cli: 'CLI-003',
    nombres: 'Mario',
    apaterno: 'Sánchez',
    amaterno: 'Morales',
    sexo: 'M',
    edad: 26,
    id_t: 2,
    estatus: 1
  },
  {
    _id: 4,
    cod_cli: 'CLI-004',
    nombres: 'Alberto',
    apaterno: 'Osako',
    amaterno: 'Murillo',
    sexo: 'M',
    edad: 29,
    id_t: 1,
    estatus: 1
  },
  {
    _id: 8,
    cod_cli: 'CLI-008',
    nombres: 'Armando',
    apaterno: 'Ramos',
    amaterno: 'García',
    sexo: 'M',
    edad: 42,
    id_t: 3,
    estatus: 1
  }
]
```

4.- Mostrar la cantidad de documentos que se encuentran registrados en la colección solicitud_cred.

Código:

```
creditos> db.solicitud_cred.countDocuments()
```

Resultado:

```
10
```

5.- Mostrar la cantidad de documentos que se encuentran en la colección solicitud_cred y su estatus es “Aceptado”.

Código:

```
creditos> db.solicitud_cred.countDocuments({ estatus: 1 })
```

Resultado:

```
4
```

6.- Mostrar el nombre completo de los clientes con el nombre del tipo de cliente con el que se encuentran registrados.

Código:

Resultado:

7.- Mostrar el código de cada solicitud con el código de cliente al que pertenece.
Código:

Resultado:

8.- Mostrar la cantidad de solicitudes que ha realizado el cliente con código “CLI-006”.
Código:

Resultado:

9.- Mostrar el promedio del monto de créditos de las solicitudes que han sido aceptadas.
Código:

Resultado:

10.- Actualizar el estatus de “0” a “1” de los clientes que su edad se encuentra entre los 25 y 50 años.
Código:

Resultado:

11.- Mostrar los folios de las solicitudes que tienen un monto entre los 80000 y 150000.
Código:

```
creditos> db.solicitud_cred.find({monto:{$gte: 80000, $lte: 150000}}, {cod_sol: 1})
```

Resultado:

```
{ _id: 4, cod_sol: 'SOL-004' },
{ _id: 5, cod_sol: 'SOL-005' },
{ _id: 7, cod_sol: 'SOL-007' },
{ _id: 9, cod_sol: 'SOL-009' }
```

12.- Mostrar el código y nombre completo de cada cliente.
Código:

```
creditos> db.cliente.aggregate([ { $project: { _id: 0, cod_cli: 1, nombre_completo: { $concat: ["$nombres", " ", "$apate
rno"," ", "$amaterno"]} } } ] )
```

Resultado:

```
{ cod_cli: 'CLI-001', nombre_completo: 'José López Pérez' },
{ cod_cli: 'CLI-002', nombre_completo: 'Daniela Ibarra Aguilar' },
{ cod_cli: 'CLI-003', nombre_completo: 'Mario Sánchez Morales' },
{ cod_cli: 'CLI-004', nombre_completo: 'Alberto Osako Murillo' },
{ cod_cli: 'CLI-005', nombre_completo: 'Gabriela Sandoval Mendoza' },
{ cod_cli: 'CLI-006', nombre_completo: 'Adriana Aguilera Sandoval' },
{ cod_cli: 'CLI-007', nombre_completo: 'Eugenia Guerrero Díaz' },
{ cod_cli: 'CLI-008', nombre_completo: 'Armando Ramos García' },
{ cod_cli: 'CLI-009', nombre_completo: 'Adriana Sánchez Moncada' },
{ cod_cli: 'CLI-010', nombre_completo: 'Eva Ramos Solis' }
```

13.- Mostar el nombre de la mujer con el monto solicitado más alto.
Código:

```
creditos> db.solicitud_cred.aggregate([ { $lookup: { from: "cliente", localField: "id_c", foreignField: "_id", as: "Clie
nte" } }, { $match: { "Cliente.sexo": "F" } }, { $sort: { monto: -1 } }, { $limit: 1 }, { $project: { _id: 0, nombre_com
pleto: { $concat: [ { $arrayElemAt: ["$Cliente.nombres", 0] }, " ", { $arrayElemAt: ["$Cliente.apaterno", 0] }] }, monto
: 1 } }] )
```

Resultado:

```
[ { monto: 100000, nombre_completo: 'Daniela Ibarra' } ]
```

14.- Mostrar la información de todos los clientes que no pertenecen al tipo “distinguido”.
Código:

```
creditos> db.cliente.aggregate([{$lookup:{from:"tipo_cli", localField:"id_t", foreignField:"_id", as: "Tipo"}}, {$match:
{"Tipo.nom_t":{$ne:"Distinguido"}}}, {$project:{_id:0, nombres: 1, apaterno: 1, amaterno: 1}}])
```

Resultado:

```
{ nombres: 'José', apaterno: 'López', amaterno: 'Pérez' },
{ nombres: 'Daniela', apaterno: 'Ibarra', amaterno: 'Aguilar' },
{ nombres: 'Mario', apaterno: 'Sánchez', amaterno: 'Morales' },
{ nombres: 'Alberto', apaterno: 'Osako', amaterno: 'Murillo' },
{ nombres: 'Adriana', apaterno: 'Aguilera', amaterno: 'Sandoval' },
{ nombres: 'Eugenia', apaterno: 'Guerrero', amaterno: 'Díaz' },
{ nombres: 'Adriana', apaterno: 'Sánchez', amaterno: 'Moncada' },
{ nombres: 'Eva', apaterno: 'Ramos', amaterno: 'Solis' }
```

15.- Mostrar el nombre del hombre con el monto solicitado más bajo.
Código:

```
creditos> db.solicitud_cred.aggregate([ { $lookup: { from: "cliente", localField: "id_c", foreignField: "_id", as: "Clie
nte" } }, { $match: { "Cliente.sexo": "M" } }, { $sort: { monto: 1 } }, {$limit:1}, { $project: { _id: 0, nombre_complet
o: { $concat: [ { $arrayElemAt: ["$Cliente.nombres", 0] }, " ", { $arrayElemAt: ["$Cliente.apaterno", 0]}, " ",  { $arra
$arrayElemAt: ["$Cliente.amaterno", 0] }] }, monto: 1 } }] )
```

Resultado:

```
[ { monto: 20000, nombre_completo: 'José López Pérez' } ]
```

16.- Mostrar el nombre de los clientes que se encuentran con una solicitud de crédito en proceso.
Código:

```
creditos> db.solicitud_cred.aggregate([
...     {
...         $match: { estatus: 3 } // Estatus 3 es "En proceso"
...     },
...     {
...         $lookup: {
...             from: "cliente",
...             localField: "id_c",
...             foreignField: "_id",
...             as: "Cliente"
...         }
...     },
...     {
...         $project: {
...             _id: 0,
...             nombre_completo: {
...                 $concat: [
...                     { $arrayElemAt: ["$Cliente.nombres", 0] }, " ",
...                     { $arrayElemAt: ["$Cliente.apaterno", 0] }, " ",
...                     { $arrayElemAt: ["$Cliente.amaterno", 0] }
...                 ]
...             }
...         }
...     }
```

Resultado:

```
{ nombre_completo: 'Mario Sánchez Morales' },
{ nombre_completo: 'Adriana Aguilera Sandoval' }
```

17.- Mostrar el nombre completo de la persona registrada en la colección cliente con la edad mayor.

Código:

```
creditos> db.cliente.aggregate([
...     {
...         $sort: { edad: -1 }
...     },
...     {
...         $limit: 1
...     },
...     {
...         $project: {
...             _id: 0,
...             nombre_completo: {
...                 $concat: ["$nombres", " ", "$apaterno", " ", "$amaterno"]
...             },
...             edad: 1
...         }
...     }
... ])
```

Resultado:

```
[ { edad: 50, nombre_completo: 'Eva Ramos Solis' } ]
```

18.- Mostrar el promedio de meses en los que los prestamos son liquidados.

Código:

```
creditos> db.solicitud_cred.aggregate([
...     {
...         $group: {
...             _id: null,
...             promedio_meses: { $avg: "$plazo_pago_meses" }
...         }
...     },
...     {
...         $project: {
...             _id: 0,
...             promedio_meses: { $round: ["$promedio_meses", 2] }
...         }
...     }
... ])
```

Resultado:

```
[ { promedio_meses: 25.1 } ]
```

19.- Mostrar la cantidad de clientes que son hombres.

Código:

```
creditos> db.cliente.aggregate([
...     {
...         $match: { sexo: "M" }
...     },
...     {
...         $count: "cantidad_hombres"
...     }
... ])
```

Resultado:

```
[ { cantidad_hombres: 4 } ]
```

20.- Mostrar la cantidad de clientes que son mujeres.

Código:

```
creditos> db.cliente.aggregate([
...     {
...         $match: { sexo: "F" }
...     },
...     {
...         $count: "cantidad_mujeres"
...     }
... ])
```

Resultado:

```
[ { cantidad_mujeres: 6 } ]
```

21.- Mostrar todos los documentos que se encuentran en la colección solicitud_cred ordenados de manera ascendente tomando como referencia de ordenamiento al campo “monto”.

Código:

```
creditos> db.solicitud_cred.aggregate([{$sort:{monto: 1}}])
```

Resultado:



```
{
  _id: 8,
  cod_sol: 'SOL-008',
  monto: 15000,
  id_c: 6,
  plazo_pago_meses: 3,
  estatus: 3
},
{
  _id: 3,
  cod_sol: 'SOL-003',
  monto: 20000,
  id_c: 1,
  plazo_pago_meses: 36,
  estatus: 2
},
{
  _id: 6,
  cod_sol: 'SOL-006',
  monto: 30000,
  id_c: 5,
  plazo_pago_meses: 36,
  estatus: 2
},
{
  _id: 10,
  cod_sol: 'SOL-010',
  monto: 40000,
  id_c: 8,
  plazo_pago_meses: 5,
  estatus: 1
},
{
  _id: 2,
  cod_sol: 'SOL-002',
  monto: 45000,
  id_c: 4,
  plazo_pago_meses: 24,
  estatus: 2
},
{
  _id: 1,
  cod_sol: 'SOL-001',
  monto: 50000,
  id_c: 10,
  plazo_pago_meses: 12,
  estatus: 1
},
{
  _id: 7,
  cod_sol: 'SOL-007',
  monto: 90000,
  id_c: 7,
  plazo_pago_meses: 24,
  estatus: 1
},
{
```

```
},
{
  _id: 4,
  cod_sol: 'SOL-004',
  monto: 100000,
  id_c: 2,
  plazo_pago_meses: 48,
  estatus: 1
},
{
  _id: 5,
  cod_sol: 'SOL-005',
  monto: 100000,
  id_c: 3,
  plazo_pago_meses: 60,
  estatus: 3
},
{
  _id: 9,
  cod_sol: 'SOL-009',
  monto: 100000,
  id_c: 9,
  plazo_pago_meses: 3,
  estatus: 2
}
]
```

22.- Mostrar el nombre completo de las personas que tienen solicitudes de créditos rechazadas. Además del nombre, en la consulta debe aparecer nombre del estatus de la solicitud y se debe omitir el campo _id.
Código:

```
creditos> db.cliente.aggregate([
...     {
...         $lookup: {
...             from: "solicitud_cred",
...             localField: "_id",
...             foreignField: "id_c",
...             as: "solicitudes"
...         }
...     },
...     {
...         $unwind: "$solicitudes"
...     },
...     {
...         $lookup: {
...             from: "tipo_estatus_c",
...             localField: "solicitudes.estatus",
...             foreignField: "_id",
...             as: "estatus_solicitud"
...         }
...     },
...     {
...         $unwind: "$estatus_solicitud"
...     },
...     {
...         $match: {
...             "estatus_solicitud.nom_tipo_ep": "Rechazado"
...         }
...     },
...     {
...         $project: {
...             "_id": 0,
...             "nombre_completo": {
...                 $concat: ["$nombres", " ", "$apaterno", " ", "$amaterno"]
...             },
...             "estatus_solicitud.nom_tipo_ep": 1
...         }
...     }
... ]);
```

Resultado:

```
{
  estatus_solicitud: { nom_tipo_ep: 'Rechazado' },
  nombre_completo: 'José López Pérez'
},
{
  estatus_solicitud: { nom_tipo_ep: 'Rechazado' },
  nombre_completo: 'Alberto Osako Murillo'
},
{
  estatus_solicitud: { nom_tipo_ep: 'Rechazado' },
  nombre_completo: 'Gabriela Sandoval Mendoza'
},
{
  estatus_solicitud: { nom_tipo_ep: 'Rechazado' },
  nombre_completo: 'Adriana Sánchez Moncada'
}
```

23. Mostrar el nombre de los tipos de clientes que tiene la base de datos.

Código:

```
creditos> db.tipo_cli.aggregate([{$lookup:{from:"cliente",localField:"id_cli",foreignField:"id_cli",as:"Cliente"}},{$unw
ind:"$Cliente"},{$project:{_id:0,nombres:"$Cliente.nombres",tipo:"$Cliente.id_t"}}])
```

Resultado:

```
{ nombres: 'José', tipo: 1 },
{ nombres: 'Daniela', tipo: 2 },
{ nombres: 'Mario', tipo: 2 },
{ nombres: 'Alberto', tipo: 1 },
{ nombres: 'Gabriela', tipo: 3 },
{ nombres: 'Adriana', tipo: 2 },
{ nombres: 'Eugenia', tipo: 1 },
{ nombres: 'Armando', tipo: 3 },
{ nombres: 'Adriana', tipo: 2 },
{ nombres: 'Eva', tipo: 1 },
{ nombres: 'José', tipo: 1 },
{ nombres: 'Daniela', tipo: 2 },
{ nombres: 'Mario', tipo: 2 },
{ nombres: 'Alberto', tipo: 1 },
{ nombres: 'Gabriela', tipo: 3 },
{ nombres: 'Adriana', tipo: 2 },
{ nombres: 'Eugenia', tipo: 1 },
{ nombres: 'Armando', tipo: 3 },
{ nombres: 'Adriana', tipo: 2 },
{ nombres: 'Eva', tipo: 1 }
```

24.- Mostrar el nombre completo de todos los clientes ordenados de la A a la Z.

Código:

```
creditos> db.tipo_cli.aggregate([{$lookup:{from:"cliente",localField:"_id",foreignField:"id_t",as:"Cliente"}},{$unwind:"
$Cliente"},{$project:{_id:0,nombres:"$Cliente.nombres",apaterno:"$Cliente.apaterno",amaterno:"$Cliente.amaterno"}},{$sor
$sort:{nombres:1}}])
```

Resultado:

```
{ nombres: 'Adriana', apaterno: 'Aguilera', amaterno: 'Sandoval' },
{ nombres: 'Adriana', apaterno: 'Sánchez', amaterno: 'Moncada' },
{ nombres: 'Alberto', apaterno: 'Osako', amaterno: 'Murillo' },
{ nombres: 'Armando', apaterno: 'Ramos', amaterno: 'García' },
{ nombres: 'Daniela', apaterno: 'Ibarra', amaterno: 'Aguilar' },
{ nombres: 'Eugenia', apaterno: 'Guerrero', amaterno: 'Díaz' },
{ nombres: 'Eva', apaterno: 'Ramos', amaterno: 'Solis' },
{ nombres: 'Gabriela', apaterno: 'Sandoval', amaterno: 'Mendoza' },
{ nombres: 'José', apaterno: 'López', amaterno: 'Pérez' },
{ nombres: 'Mario', apaterno: 'Sánchez', amaterno: 'Morales' }
```

25.- Obtener el promedio del monto de las solicitudes de créditos agrupadas por el tipo de estatus de solicitud.

Código:

```
creditos> db.solicitud_cred.aggregate([{$group:{_id:"$estatus",promedio_monto:{$avg:"$monto"}}},{$project:{_id:0,estatus
estatus: "$_id",promedio_monto:1}}])
```

Resultado:

```
{ promedio_monto: 57500, estatus: 3 },
{ promedio_monto: 48750, estatus: 2 },
{ promedio_monto: 70000, estatus: 1 }
```

26.- Mostrar el nombre de los clientes que tienen entre 20 y 30 años y tienen solicitudes de crédito aprobadas. Mostrar también el monto de dichas solicitudes

omitiendo en el resultado el campo _id.

Código:

```
creditos>
... db.cliente.aggregate([
...     {
...         $lookup: {
...             from: "solicitud_cred",
...             localField: "_id",
...             foreignField: "id_c",
...             as: "solicitudes"
...         }
...     },
...     {
...         $unwind: "$solicitudes"
...     },
...     {
...         $match: {
...             "edad": { $gte: 20, $lte: 30 },
...             "solicitudes.estatus": 1
...         }
...     },
...     {
...         $project: {
...             "_id": 0,
...             "nombres": 1,
...             "solicitudes.monto": 1
...         }
...     }
... ]);
```

Resultado:

```
[ { nombres: 'Daniela', solicitudes: { monto: 100000 } } ]
```

27.- Mostrar la cantidad de clientes registrados agrupados por edad.

Código:

```
creditos>
... db.cliente.aggregate([
...     {
...         $group: {
...             _id: "$edad",
...             cantidad_clientes: { $sum: 1 }
...         }
...     },
...     {
...         $project: {
...             _id: 0,
...             edad: "$_id",
...             cantidad_clientes: 1
...         }
...     }
... ]);
```

Resultado:

```
{ cantidad_clientes: 1, edad: 29 },
{ cantidad_clientes: 1, edad: 26 },
{ cantidad_clientes: 1, edad: 42 },
{ cantidad_clientes: 1, edad: 33 },
{ cantidad_clientes: 1, edad: 40 },
{ cantidad_clientes: 1, edad: 45 },
{ cantidad_clientes: 1, edad: 50 },
{ cantidad_clientes: 1, edad: 18 },
{ cantidad_clientes: 1, edad: 19 },
{ cantidad_clientes: 1, edad: 30 }
```

28.- Aprobar la solicitud con folio “SOL-002”.

Código:

```
creditos> db.solicitud_cred.updateOne(
...     { "cod_sol": "SOL-002" },
...     { $set: { "estatus": 1 } }
... );
```

Resultado:

```
_id: 2,
cod_sol: 'SOL-002',
monto: 45000,
id_c: 4,
plazo_pago_meses: 24,
estatus: 1
```

29.- Mostrar el código de cada uno de los clientes con el tipo de cliente que tiene asignado.

Código:

```
creditos> db.cliente.aggregate([
...     {
...         $lookup: {
...             from: "tipo_cli",
...             localField: "id_t",
...             foreignField: "_id",
...             as: "TipoCliente"
...         }
...     },
...     {
...         $project: {
...             _id: 0,
...             cod_cli: 1,
...             nom_t: { $arrayElemAt: ["$TipoCliente.nom_t", 0] }
...         }
...     }
... ]);
...
```

Resultado:

```
{ cod_cli: 'CLI-001', nom_t: 'Inicial' },
{ cod_cli: 'CLI-002', nom_t: 'Frecuente' },
{ cod_cli: 'CLI-003', nom_t: 'Frecuente' },
{ cod_cli: 'CLI-004', nom_t: 'Inicial' },
{ cod_cli: 'CLI-005', nom_t: 'Distinguido' },
{ cod_cli: 'CLI-006', nom_t: 'Frecuente' },
{ cod_cli: 'CLI-007', nom_t: 'Inicial' },
{ cod_cli: 'CLI-008', nom_t: 'Distinguido' },
{ cod_cli: 'CLI-009', nom_t: 'Frecuente' },
{ cod_cli: 'CLI-010', nom_t: 'Inicial' }
```

30.- Mostrar el nombre completo de los clientes que pagaran sus créditos en un plazo mayor a 5 meses, pero menor a 20. Considere que para esta consulta solo se deben considerar las solicitudes que tiene un estatus de “aceptada”.

Código:

```
itos> db.cliente.aggregate([
    {
        $lookup: {
            from: "solicitud_cred",
            localField: "_id",
            foreignField: "id_c",
            as: "Solicitudes"
        }
    },
    {
        $unwind: "$Solicitudes"
    },
    {
        $match: {
            "Solicitudes.plazo_pago_meses": { $gt: 5, $lt: 20 },
            "Solicitudes.estatus": 1
        }
    },
    {
        $project: {
            _id: 0,
            nombre_completo: {
                $concat: ["$nombres", " ", "$apaterno", " ", "$amaterno"]
            }
        }
    }
]);
```

Resultado:

```
[ { nombre_completo: 'Eva Ramos Solis' } ]
```

31.- Mostrar la edad promedio de los distintos tipos de clientes que se tienen.

Código:

```
creditos> db.cliente.aggregate([{ $lookup: { from: "tipo_cli", localField: "id_t", foreignField: "_id", as: "tipoCliente
" } }, { $project: { _id: 0, edad: 1, tipo_cliente:{$arrayElemAt:["$tipoCliente.nom_t",0]}}},{$group:{_id:"$tipo_cliente
",prom:{$avg:"$edad"}}}])
```

Resultado:

```
{ _id: 'Distinguido', prom: 41 },
{ _id: 'Inicial', prom: 35.75 },
{ _id: 'Frecuente', prom: 26.75 }
```

32.- Mostrar el nombre de los clientes que no tienen solicitudes de créditos rechazadas.

Código:

```
creditos> db.cliente.aggregate([{ $lookup: { from: "solicitud_cred", localField: "_id", foreignField: "id_c", as: "soli
itudes" } }, { $match: { "solicitudes.estatus": { $ne: 2 } } }, { $project: { _id: 0, nombres: 1, apaterno: 1, amaterno:
 1 } } ])
```

Resultado:

```
{ nombres: 'Daniela', apaterno: 'Ibarra', amaterno: 'Aguilar' },
{ nombres: 'Mario', apaterno: 'Sánchez', amaterno: 'Morales' },
{ nombres: 'Alberto', apaterno: 'Osako', amaterno: 'Murillo' },
{ nombres: 'Adriana', apaterno: 'Aguilera', amaterno: 'Sandoval' },
{ nombres: 'Eugenia', apaterno: 'Guerrero', amaterno: 'Díaz' },
{ nombres: 'Armando', apaterno: 'Ramos', amaterno: 'García' },
{ nombres: 'Eva', apaterno: 'Ramos', amaterno: 'Solis' }
```

33.- Mostrar el nombre los clientes que se encuentran con una solicitud en proceso. Entendiendo que en esta consulta no se debe usar la función “igual” sino “en”.

Código:

```
creditos> db.cliente.aggregate([{ $lookup: { from: "solicitud_cred", localField: "_id", foreignField: "id_c", as: "solic
itudes" } }, { $match: { "solicitudes.estatus": { $in: [3] } } }, { $project: { _id: 0, nombres: 1, apaterno: 1, amatern
amaterno: 1 } } ])
```

Resultado:

```
{ nombres: 'Mario', apaterno: 'Sánchez', amaterno: 'Morales' },
{ nombres: 'Adriana', apaterno: 'Aguilera', amaterno: 'Sandoval' }
```

34.- Agregar un nuevo documento a la colección solicitud_cred llamado “Liquidado”.

Código:

```
creditos> db.tipo_estatus_c.insertOne({ "_id": 4, "nom_tipo_ep": "Liquidado" });
```

Resultado:

```
{ _id: 1, nom_tipo_ep: 'Aceptado' },
{ _id: 2, nom_tipo_ep: 'Rechazado' },
{ _id: 3, nom_tipo_ep: 'En proceso' },
{ _id: 4, nom_tipo_ep: 'Liquidado' }
```

35.- Actualizar el estatus de las solicitudes SOL-006 y SOL-007 de “Aceptado” a“Liquidado”.

Código:

```
creditos> db.solicitud_cred.updateOne({estatus:2, cod_sol:"SOL-006"},{$set:{estatus:4}})
... db.solicitud_cred.updateOne({estatus:1, cod_sol:"SOL-007"},{$set:{estatus:4}})
```

Resultado:

```
  _id: 6,
  cod_sol: 'SOL-006',
  monto: 30000,
  id_c: 5,
  plazo_pago_meses: 36,
  estatus: 4
},
{
  _id: 7,
  cod_sol: 'SOL-007',
  monto: 90000,
  id_c: 7,
  plazo_pago_meses: 24,
  estatus: 4
},
```

36.- Mostrar el código de las solicitudes con el nombre del estatus que tienen. (Esta consulta es similar a una de las anteriores, su ejecución nuevamente es necesaria para ver que solicitudes tienen el nuevo estatus “Liquidado”.

Código:

```
creditos> db.solicitud_cred.aggregate([{ $lookup: { from: "tipo_estatus_c", localField: "estatus", foreignField: "_id", as: "Estatus" } }, { $project: { _id: 0, cod_sol: 1, estatus:{$arrayElemAt:["$Estatus.nom_tipo_ep",0]}}}])
```

Resultado:

```
{ cod_sol: 'SOL-001', estatus: 'Aceptado' },
{ cod_sol: 'SOL-002', estatus: 'Aceptado' },
{ cod_sol: 'SOL-003', estatus: 'Rechazado' },
{ cod_sol: 'SOL-004', estatus: 'Aceptado' },
{ cod_sol: 'SOL-005', estatus: 'En proceso' },
{ cod_sol: 'SOL-006', estatus: 'Liquidado' },
{ cod_sol: 'SOL-007', estatus: 'Liquidado' },
{ cod_sol: 'SOL-008', estatus: 'En proceso' },
{ cod_sol: 'SOL-009', estatus: 'Rechazado' },
{ cod_sol: 'SOL-010', estatus: 'Aceptado' }
```

Glosario.

Función	Definición

EVALUACIÓN DEL SEGUNDO PARCIAL
BASES DE DATOS PARA CÓMPUTO EN LA NUBE
ADMINISTRACIÓN DE BASES DE DATOS PARA CÓMPUTO EN LA NUBE

$match	solo los documentos que cumplen esta condición
$group	Agrupa documentos con base a un campo y permite hacer cálculos como contar elementos o sacar promedios.
$sum	Se usa con $group para sumar valores. Es como un acumulador.
$avg	Calcula el promedio de los valores en un grupo.
$sort	Ordena los resultados de menor a mayor
$project	Sirve para mostrar u ocultar ciertos campos en el resultado
$lookup	Es como un JOIN. Una dos colecciones que tiene un campo en común.
$unwind	Si un campo tiene una matriz, lo separa en documentos individuales.
$addFields	Agregue nuevos campos al resultado, ya sea con valores fijos o calculados.
$set	se usa para actualizar documentos en update
$arrayElemAt	Obtiene un elemento en una posición específica de una matriz
$in	Busca valores que estén dentro de una lista de opciones.
$gte, $lte, $gt, $lt	>=($gte), <=($lte), >($gt), <($lt)
$exists	Revisa si un campo existe en un documento o no.
$push	Agrega un valor a una matriz dentro de un documento
$pull	Elimina elementos de una matriz dentro de un documento.
$regex	Permite buscar texto con expresiones regulares